

There are different strategies to handle such scenarios. However, it all boils down to the way the domain is decomposed into sub-domains. Often, the sub-domains are aligned to the lines of business. For instance, the catalogue service, the order service, and the fulfilment service are some of the probable sub-domains of an e-commerce domain. There are also secondary sub-domains that are not directly connected to the business, but are important, such as accounts. And we have sub-domains that are technology-specific, like messaging systems. A careful demarcation of the sub-domain boundaries is vital to get some benefit out of DDD.

*Who does the domain decomposition? It looks as if it is beyond the scope of a software designer.*

Yes. The domain decomposition is the responsibility of the management team. The senior solution architects are usually part of such a management team. Once the decomposition is finalised and sub-domains are identified, the engineering teams take over the responsibility to model and implement them. Speaking technically, instead of delivering one single monolith enterprise system, the engineering department delivers a set of smaller services which are now popularly known as microservices.

*Does this mean that there will be no overlap between two sub-domains?*

The degree of overlap between sub-domains is directly related to the nature of the business and the nature of decomposition. At one extreme, there is no overlap, which is desired. At another extreme, there is a lot of overlap that defeats the very purpose of decomposition. Both are very rare. Often, we see sub-domains with a small degree of overlap. Still, teams are advised to deal with their sub-domains independent of others, even if it results in duplicate effort. In the long run, a set of loosely coupled services are more beneficial.

*Other than high-level concepts and best practices, does DDD offer any handy patterns that are easy to follow and replicate?*

Of course, yes! Entities, value objects (VOs), data transfer objects (DTOs), aggregates, and repositories are very popular though a good number of people may not really be aware of the fact that they are DDD patterns. An entity is a domain object that can be uniquely identified, whereas a VO is a domain object that does not have a unique identity. A DTO is an object that is transferred across the domain layer. An aggregate is a cluster of domain objects with a root, and a repository is an abstraction of the aggregate store.

*Is the entity of DDD the same as the entity of ORM, E-R models, etc?*

They are not exactly the same. An entity in DDD is not necessarily associated with any persistence. They may also exist only during runtime. The point is that any domain object that is uniquely identified is called an entity. In other words, the state of an entity can change over a period of time, but it can still be identified uniquely. For example, an *order* in an e-commerce system is identified by something like *order-id*, though the value of the order and status of the order keeps changing.

*An employee with an employee-id, a customer with a customer-id, an account with account-number, and a book with ISBN are all entities, then! They may or may not be persisted. When persisted, they may be mapped to a record in the RDBMS, or a document in NoSQL, etc. Entities that have relations with other entities form aggregates. Can the entities also have behaviour?*

Of course, they can! Entities are first-class domain objects. They can have *behaviour* besides the *state*. For example, an *order* object may have operations like *applyDiscount()*, *cancel()*, *deliver()*, etc. For many, this looks a bit odd since somehow they are under the impression that an entity is just a database record without any operations. And that is a misplaced understanding.

*Well, a domain object with a unique identity is an entity, and a domain object without a unique identity is a value object. Is that right?*

That's not the way to look at it. A value object is identified by looking at the whole of the object, not just one property. For example, two event objects in a UI system are indistinguishable if the type, time, and coordinates of the event are exactly the same. The value objects must be immutable, since any change in the value of the object makes it unrecognisable. Usually, it is encouraged to model properties of primitive types like phone numbers, email, addresses, etc, as value objects. Like the entities, the VOs also may or may not be persisted.

*How are the entities and value objects persisted? Do they use data access objects?*

One of the hallmarks of the DDD is that the domain model is technology-agnostic. In other words, the objects in the domain layer do not really know about the outside world. Even the storing and retrieving of the domain objects is done through an abstraction called repository. A repository gives an impression as if it is in-memory storage of the domain objects. Some of the usual methods of a repository include *save()*, *find()*, *update()*, *delete()*, etc.

Unlike DTOs, which are more inclined towards databases, repositories may be backed by anything such as databases, file systems, REST endpoints, etc. A notable